

FoodLink An Application For Redistributing Surplus Food To Reduce Waste

Blaudschun Beauvoir, Deylis Cano Morera, Monisha Kummari, & Akul Singh

CEN5011 RVC 1258: Advanced Software Engineering

Dr. Charlyne Walker

December 7, 2025

FoodLink An Application For Redistributing Surplus Food To Reduce Waste

This project focuses on developing an application that helps reduce food waste by connecting food providers with beneficiaries in need. Restaurants, grocery stores, and households frequently dispose of surplus food, while local food banks and shelters often face shortages. Our application will act as a platform where food providers can post excess food, and organizations or individuals can claim or request it, promoting resource sharing and reducing waste.

FoodLink's Purpose

This project is designed to cater towards different groups of people in terms of end users. It is intended to provide usability for both food banks/shelters as well as those willing to donate food, whether that be individuals or businesses such as restaurants.

The Problem

The application is a food redistribution platform designed to reduce waste by connecting donors (restaurants, groceries, households) with recipients (food banks, shelters, individuals). The background stems from the ongoing issue of massive food waste coexisting with food insecurity. The U.S. Department of Agriculture (n.d) notes that around “30-40 percent of the food supply” is wasted, which is an issue when it could instead be going to families in need. The main problem is the lack of visibility, speed, and fairness in matching surplus food with those in need, resulting in spoilage, inequitable distribution, and barriers for smaller organizations and households to participate.

How FoodLink is Unique

Our tool is different in comparison to other applications such as MealConnect in that this aims towards balancing distribution as well. MealConnect mainly operates by letting users post their food and sends it to the closest partner in their network (Feeding America, n.d.). For example, while MealConnect primarily serves large organizations, FoodLink also ensures fair distribution for smaller groups and households. FoodLink also intends to allow food banks and shelters to note which foods they are desperately in need of and works towards making sure they receive an adequate amount.

FoodLink's Implications

The implications of FoodLink include reducing food waste and improving food access by connecting donors with those in need. It may change how food banks and shelters manage donations, require training, and create reliance on the platform. Key concerns involve food safety, privacy, fairness in distribution, and compliance with food safety and donation laws. Overall, the tool has technical, legal, and social impacts that go beyond just the software.

Important Contextual Issues

Legal requirements, such as compliance with the Good Samaritan Food Donation Act (U.S. Department of Agriculture, 2020), must be observed to ensure that donated food is safe for consumption. Logistics challenges, including storage, preservation, and transportation, must be addressed. Nutritional concerns should be considered to ensure donations meet dietary and cultural needs. Food safety is critical to prevent spoilage or contamination during redistribution.

Proposed Schedule

The project will be developed over a period of ten weeks using an Agile Scrum methodology. The schedule is divided into five sprints, each lasting two weeks. This approach allows the team to build the application incrementally, beginning with setup and database design, moving into backend and frontend development, and finishing with system integration, testing, and final delivery.

For the project, we currently intend to utilize either Python or Java for the backend, a database through MySQL or PostgreSQL, and a front-end component that will mainly use HTML, CSS, and JavaScript.

User Story Backlog

The following user stories were developed in order to break down the development of the project into smaller pieces and understand different use case scenarios.

Feature 1: Registration

- As a user, I want to be able to create a username and password, so that I can manage my food donations and requests.

Table 1

User Story: Registering an Account

Acceptance Criteria	If	When	Then
AC 1	I am a user	I visit the website	I should see a register button

AC 2		I click on the button	I should see a text box to enter a username, password.
AC 3		I enter all my information	It should tell me if the username and password I have entered are acceptable.

- As a user, I want to input my email, so that I have a way of verifying my account

Table 2

User Story: Verifying an Account

Acceptance Criteria	If	When	Then
AC 1	I am a user	I visit the website	I should see a registration button
AC 2		I click on the button	I should see a text box to enter an email
AC 3		I enter an email	I should be notified if the email is in use or not

- As a user, I want a visible error message to display if I am missing or have invalid information, so that I can correct it

Table 3*User Story: Invalid Entry*

Acceptance Criteria	If	When	Then
AC 1	I am a user	I visit the website	I should see a registration button
AC 2		I click on the button	I should see text boxes to enter information
AC 3		I enter my information	I should be notified if any information is unacceptable, incomplete, or already in use

- As a user, I want to receive a confirmation email so that I know my account is officially created

Table 4*User Story: Confirmation Email*

Acceptance Criteria	If	When	Then
AC 1	I am a user	I finish inputting my information	I should see a register button

AC 2		I click on the button	I should see a message saying a confirmation email was sent to my account
AC 3		I visit my email inbox	I see an email that tells me an account was created and to confirm my email

Feature 2: Login

- As a user, I want to be able to enter my existing username and password, so that I can access my account.

Table 5

User Story: Logging In

Acceptance Criteria	If	When	Then
AC 1	I am a registered user	I visit the website	I should see a login button
AC 2		I click on the button	I should be taken to the login page

AC 3		I am on the login page	I should see text boxes to input my username and password
------	--	------------------------	---

- As a user, I want to be able to reset my password, so that I can make sure my account is secure.

Table 6

User Story: Resetting an Account Password

Acceptance Criteria	If	When	Then
AC 1	I am a registered user	I am on the login page	I should see a button that says “Forgot password?”
AC 2		I click on that button	I should be taken to a page with a textbox that allows me to enter my username or email
AC 3		I enter my username or email	I receive an email in my inbox giving me

			instructions on how to change my password
--	--	--	---

- As a user, I want a visible error message to display so that I know if my username or password is wrong.

Table 7

User Story: Incorrect Login

Acceptance Criteria	If	When	Then
AC 1	I am a registered user	I type in my username	I should be given an error if I typed it incorrectly or it doesn't exist
AC 2		I type in my password	I should be given an error if it was typed incorrectly
AC 3		I click the login button but am unable to login	I should be given an error saying my username and password don't match

- As a user, I want an option to save my login details, so that I don't have to login to the website every time.

Table 8

User Story: Saving Login Information

Acceptance Criteria	If	When	Then
AC 1	I am a registered user	I visit the login page	I should see a check box that says "Save login details"
AC 2		I click on the checkbox	A check mark appears notifying me
AC 3		I visit the website again	My login information should be filled in without me typing anything

- As a user, I want to be able to see a sign up button on the login page so that if I don't have an account yet, I am able to easily register for one.

Table 9

User Story: Signing Up for an Account

Acceptance Criteria	If	When	Then
---------------------	----	------	------

AC 1	I am a user	I visit the website	I should see a button to login
AC 2		I click on the button	I should be taken to the login page
AC 3		I am on the login page	I should see a button that says “Register” if I don’t have an account

Feature 3: Profile

- As a user, I want to be able to access a profile dashboard so that I can see my account details.

Table 10

User Story: Going to Profile

Acceptance Criteria	If	When	Then
AC 1	I am a user	I go to the website	I should see a button to login
AC 2		I click on the button	I should be taken to the login page

AC 3		I enter my credentials and click the login button	I should see a Profile page containing my account details
------	--	---	---

- As a user, I want to be able to access my profile dashboard so that I can change my account details.

Table 11

User Story: Changing Profile Details

Acceptance Criteria	If	When	Then
AC 1	I am a user	I login to the website	I should see a profile page
AC 2		I click on the button	I should be taken to the edit details page
AC 3		I enter my new details and click save	I should see a Profile page containing my new details

Feature 4: Food Donation

- As a donor, I want to be able to upload a post with the food I have available so I can donate it

Table 12*User Story: Donating Food*

Acceptance Criteria	If	When	Then
AC 1	I am a donor	I click on the donate button	I should see a donate page
AC 2		I enter the food I have available and click enter	I should be taken to a summary page
AC 3		I click confirm	I should see a page showing the food is available for donation

- As a donor, I want to be able to edit a post of mine because I put the details for some food so others aren't misled by it

Table 13*User Story: Updating Donated Food Post*

Acceptance Criteria	If	When	Then
AC 1	I am a donor	I click on the my donations button	I should see a page with all my donations

AC 2		I click on a specific donation	I should be taken to a summary page
AC 3		I click the edit button	I should see boxes for each food to edit and update the details
AC 4		I click the update button	I see the updated page for the food donation with a note that it has been updated

Feature 5: Food Requests

- As a Food Bank in need of food, I want to be able to look at the food available to me so that I can accept whatever I need.

Table 14

User Story: Finding Available Food

Acceptance Criteria	If	When	Then
AC 1	I am a Food Bank	I click on the receive food button	I should see an available food page

AC 2		I click on the food post for the food I want to grab	I should be taken to a summary page
AC 3		I click confirm	I should see a page showing the food will now be allocated for me

- As a Food Bank in need of a specific type of food, I want to be able to only look at that specific type of food available to me so that I can accept whatever I need.

Table 15

User Story: Filtering Available Food

Acceptance Criteria	If	When	Then
AC 1	I am a Food Bank	I click on the receive food button	I should see an available food page
AC 2		I click on the filter button	I should see a pop up that limits which foods are shown to me

AC 3		I click the checkboxes for the specific foods I want	I should see a page showing only the available foods who satisfy the constraints
------	--	--	--

Feature 6: Accessing Admin Panel

- As a moderator of FoodLink, I want to be able to access the logs so that I can see the history of posts

Table 16

User Story: Accessing Logs

Acceptance Criteria	If	When	Then
AC 1	I am a moderator	I login in as an admin	I should see a unique admin page
AC 2		I click on the logs button	I should see a page with all the logs
AC 3		I click on a specific log	I should see a log page with all the exact log details

- As a moderator of FoodLink, I want to be able to approve posts of food donations so those in need can see them

Table 17*User Story: Approving Donations*

Acceptance Criteria	If	When	Then
AC 1	I am a moderator	I go to the admin page	I should see a pending donations button
AC 2		I click on the pending donations button	I should see a page with all the pending donations
AC 3		When I click on a pending donation	I should see a page with all the details of the donation
AC 4		When I click on the approve button	The donation is approved and those in need can now see it

- As a moderator of FoodLink, I want to be able to reject posts of suspicious food donations so that it isn't harmful for others

Table 18*User Story: Rejecting Donations*

Acceptance Criteria	If	When	Then
AC 1	I am a moderator	I go to the admin page	I should see a pending donations button
AC 2		I click on the pending donations button	I should see a page with all the pending donations
AC 3		When I click on a pending donation	I should see a page with all the details of the donation
AC 4		When I click on the reject button	The donation is rejected and is deleted from the database and the user who sent the donation is notified

- As a moderator of FoodLink, I want to be able to deactivate users who post suspicious food donations so that it isn't harmful for others

Table 19

User Story: Deactivating Users

Acceptance Criteria	If	When	Then
AC 1	I am a moderator	I go to the admin page	I should see a users list button
AC 2		I click on the users list button	I should see a page with all the users
AC 3		When I click on a user	I should see a page with all the details of the users
AC 4		When I click on the deactivate button	The user is deactivated and is emailed about it

User Story Allocation

To determine user story allocation, we will use an iterative and priority-driven process aligned with the Agile Scrum methodology. First we will place all the user stories collected in the backlog. From the backlog we will allocate the stories across five sprints. If a user story is dependent on another it will be scheduled earlier. Features that are more critical to the systems core or to the users will be in earlier sprints compared to optional features or ones that do not need to be there in the initial release of the application. Stories that are more complex will be broken down into simpler stories and be allocated to more developers. Stories will be assigned to team members based on technical background to ensure high quality results and efficiency:

- *Dependencies*: Database and backend tasks must be completed before frontend and user flows.
- *Complexity*: Simpler stories (viewing donations) assigned earlier; complex workflows (claims, restrictions) later.
- *Team Strengths*: Each member focuses on areas matching their skills (backend, frontend, testing, coordination).

Sprint Period

The project timeline will span 10 weeks, divided into five sprints. Each sprint period will have a duration of 2 weeks. User stories will be allocated to them based on the feasibility, considering the complexity of each. Sprints that can not be completed in the assigned sprint will be allocated into the next sprint to ensure they are completed and to have manageable workloads.

Table 20

Sprint Schedule

Sprint	Weeks	User Stories	Tasks
1	1-2	Registration	- Decide on framework and tools (backend, frontend, database) - Set up project repository - Design and implement initial database schema - Build backend skeleton - Create simple “Hello World” frontend to verify setup
2	3-4	Login	- Connect backend to database

			- Develop initial frontend skeleton (navigation, basic pages) - Implement user registration and login - Implement Security via JSON Web Tokens (JWT) - Add ability for donors to post donations (basic form) - Partition database for donors and recipients
3	5-6	Food Donation	- Implement donation browsing for recipients (food banks, shelters, individuals) - Add filtering (by type, expiration, availability) - Develop claim/request functionality - Apply rules for individual donors (sealed/unopened items) - Add basic admin monitoring dashboard
4	7-8	Food Requests	- Connect frontend and backend fully (JSON APIs) - Enable editing and canceling of donations - Implement fair-share distribution logic (balance among recipients) - UI polish (improve look and usability) - Perform full system testing - Prepare documentation and final demo
5	9-10	Profile	- Implement profile dashboard (view and edit account details) - Add user role management (donor, recipient, moderator) - Build admin panel features (approve/reject donations,

			deactivate users) - Integrate notifications (confirmation emails, status updates) - Final integration of frontend and backend - Conduct end-to-end testing - Polish UI/UX and accessibility - Finalize documentation and prepare presentation/demo
--	--	--	---

Project Progression

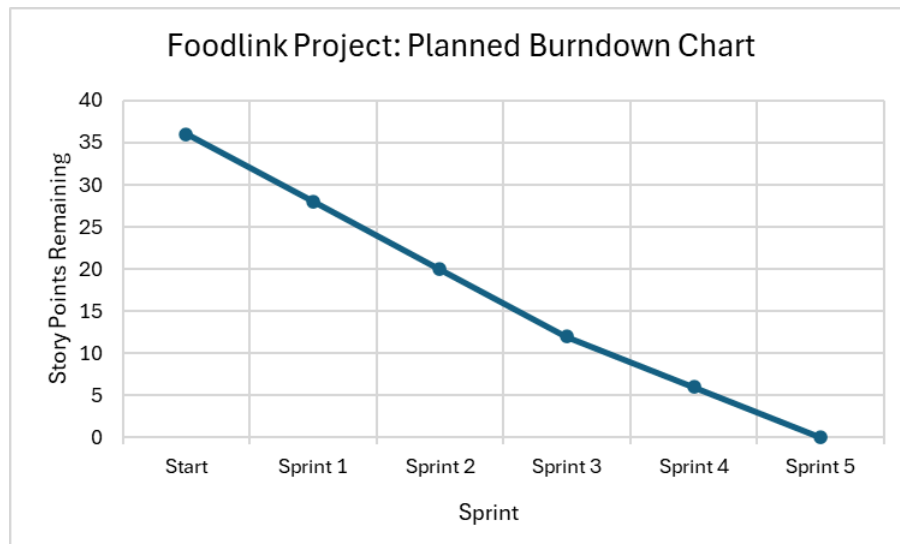
We estimate 36 total story points across all user stories. Points will be distributed across sprints as follows:

- Sprint 1: 8 points
- Sprint 2: 8 points
- Sprint 3: 8 points
- Sprint 4: 6 points
- Sprint 5: 6 points

The burndown chart will also display a steady decline from 36 → 0 across the 10 weeks, with the ideal line being diagonal and actual progress tracked after each sprint review.

Figure 1

FoodLink Project: Planned Burndown Chart



Team Roles and Responsibilities

Our team will be splitting the roles and responsibilities mainly in terms of Front-end and Back-end development. Outside of that, the members will share mixed roles across project management, testing, as well as deployment of the application.

Table 21

Team Roles

Area	Responsibilities	Names
Project Management	Plan sprints, track progress, maintain backlog, document decisions.	Rotating role (all members take turns)
Backend Development	Contribute to database schema, handle authentication and data storage.	Blaudschun Beauvoir, Deylis Cano Morera

Frontend Development	Develop UI components, connect frontend with backend, ensure accessibility and usability.	Akul Singh, Monisha Kummari
Testing & QA	Design and run test cases, verify acceptance criteria, report and fix defects.	Rotating role (all members take turns)
Integration & Deployment	Integrate backend and frontend, troubleshoot issues.	Rotating role (all members take turns)

Design

In this section, we describe the design process utilized for Food Link as well as the inner workings for the application.

Definition of Design

In software engineering, design refers to the systematic process of defining a system's architecture, components, interfaces, and data to satisfy specified requirements (Sommerville, 2016). It serves as the bridge between requirements analysis and implementation, ensuring that the system structure supports functional and non-functional needs. Pressman and Maxim (2020) describe software design as a “modeling activity that translates requirements into a blueprint for constructing the software,” emphasizing abstraction, modularity, and separation of concerns.

In other words, design transforms what the system must do (requirements) into how it will do it (structure and behavior). It includes both architectural design (overall framework) and detailed design (module-level logic and data structures).

Design Process

Our team followed an Agile-Scrum design process guided by iterative development and continuous feedback. Following the stages described by Sommerville (2016), we reviewed requirements, modeled system architecture, and refined the design through cycles of collaboration and prototype evaluation. The process followed these key steps:

1. **Requirement Review:** Team members analyzed the user stories and acceptance criteria from the introduction to identify system functionalities and dependencies.

2. **Architecture Planning:** The group discussed various frameworks and selected a lightweight, scalable architecture combining a Flask backend and a React frontend.
3. **Data Modeling:** Database entities (e.g., users, donations, requests) were outlined and normalized for relational storage in MySQL.
4. **UI/UX Design:** Simple mockups and layout drafts were discussed for major pages (registration, donation listing, admin dashboard).
5. **Refinement and Documentation:** The team finalized diagrams, updated design rationale, and prepared for the implementation phase.

Security Design

To ensure data protection and secure access, FoodLink incorporates several security features. Authentication is managed using JSON Web Tokens (JWT), which prevent unauthorized access to restricted endpoints. All data exchanges between frontend and backend occur over HTTPS, and user passwords are stored in the database using encryption. Role-based access control ensures that administrators, donors, and recipients have permissions appropriate to their roles.

System Architecture

FoodLink uses a three-tier architecture that separates the user interface, business logic, and data layers. This separation enhances scalability, simplifies maintenance, and improves the clarity of system interactions.

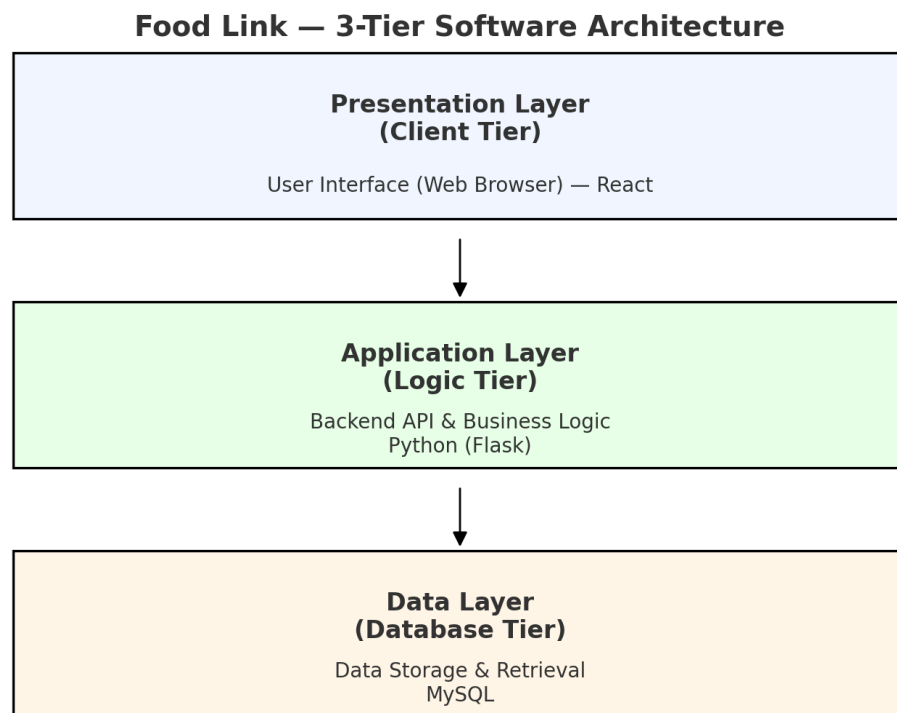
- **Frontend (React):** Manages the user interface and handles user input for login, registration, donation, and request features.

- Backend (Flask): Processes user requests, handles authentication, and enforces business logic through RESTful APIs.
- Database (MySQL): Stores persistent data including users, donations, requests, and admin activity logs.

The architecture also enforces security through token-based authentication and server-side validation to prevent unauthorized access and data leakage.

Figure 2

FoodLink Software Architecture



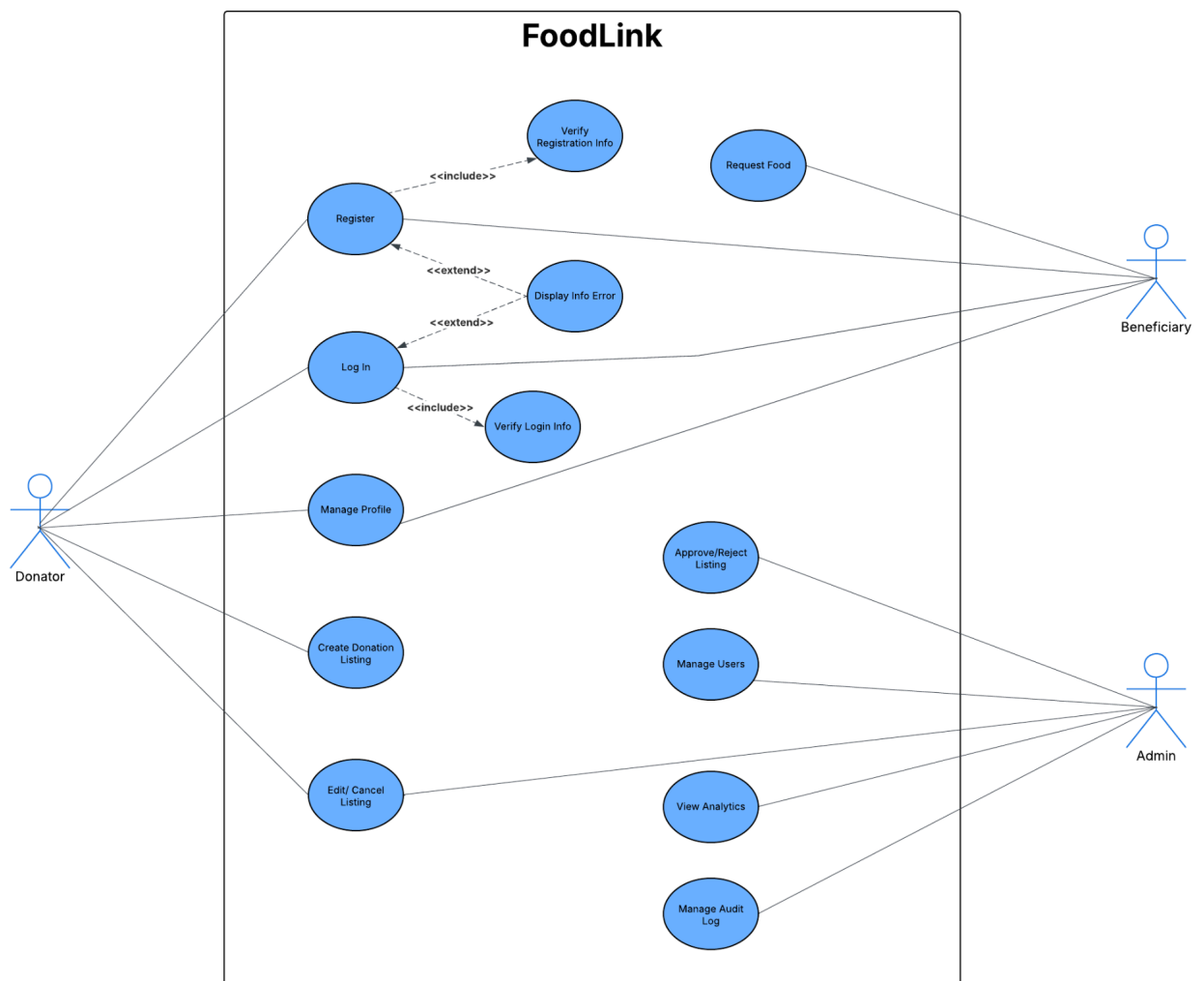
Use Case Diagram

The FoodLink system supports three primary user types: Donors, Recipients, and Administrators.

- Donors can register, log in, and post available food items.
- Recipients can browse or request donations and filter by category or availability.
- Administrators review, approve, or reject donations and manage user accounts.

Figure 3

Use Case Diagram for FoodLink



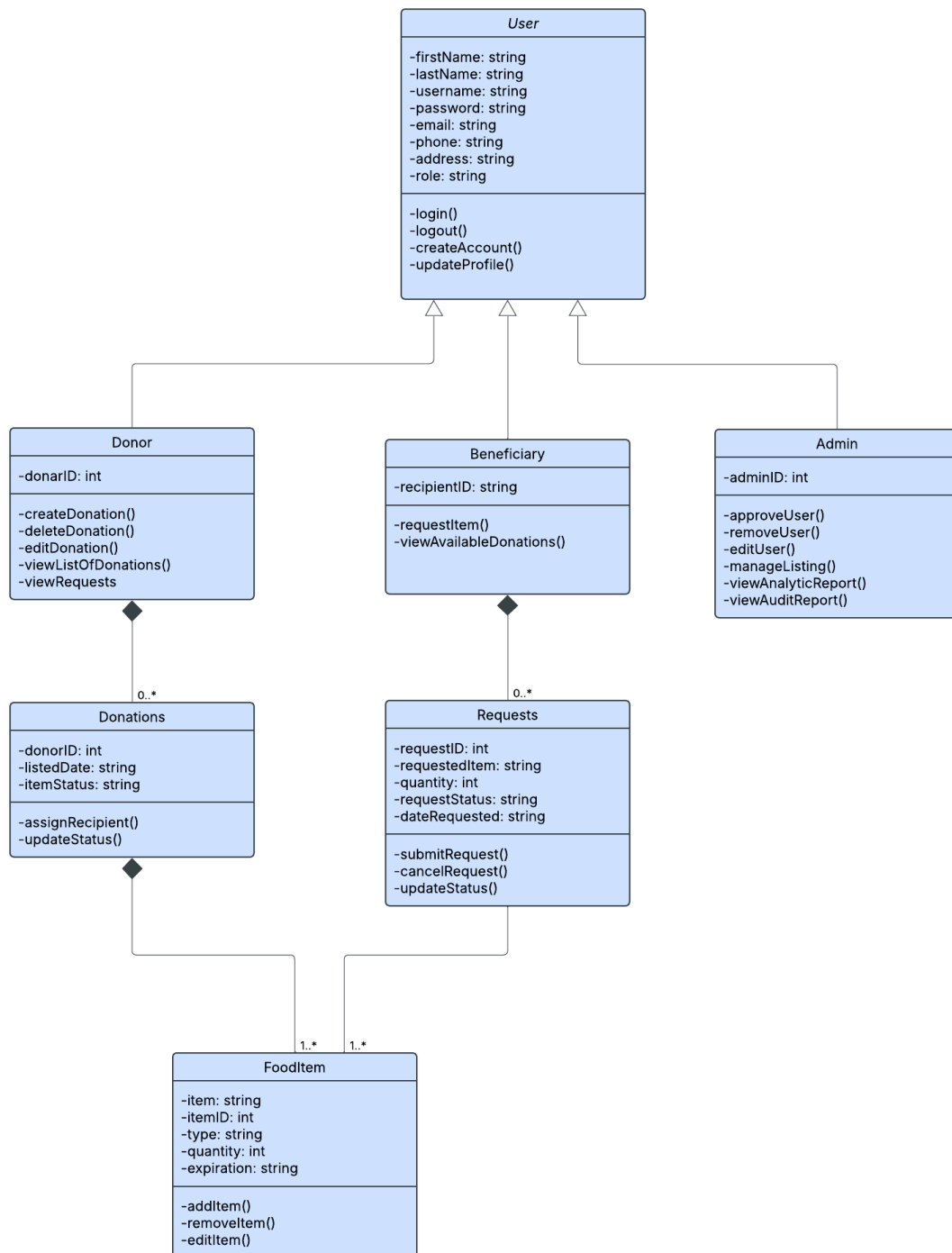
Class Diagram

The class diagram represents the structure of the system's key components and their relationships.

Table 22

Class Diagram Table

Class	Description
User	Represents registered individuals; attributes include ID, name, email, role, and password.
Donation	Stores food donation details such as item type, quantity, expiration, and donor ID.
Request	Manages food requests and links to both user and donation records.
Admin	Extends User with privileges to approve/reject donations and deactivate accounts.
DatabaseConnection	Handles communication with the MySQL database.

Figure 4*Class Diagram for FoodLink*

Sequence Diagrams

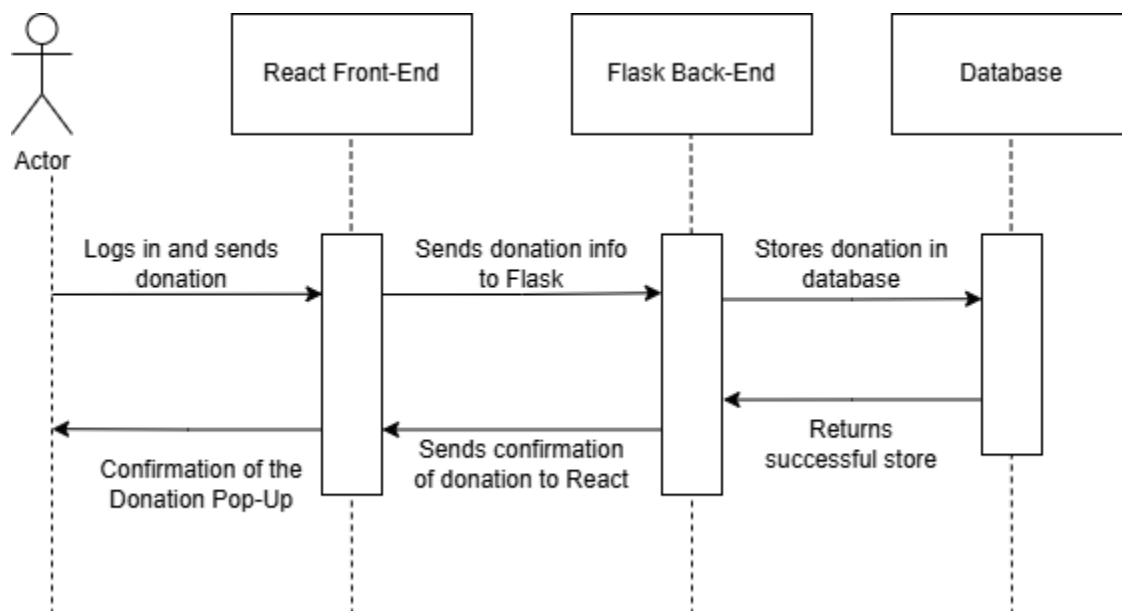
The sequence diagram below outlines the process of a donor posting a food donation, showing how the system components interact.

Steps:

1. Donor logs in via the React frontend.
2. The frontend sends a POST request to the Flask API with donation details.
3. Flask validates the input and stores it in the database.
4. Flask returns a confirmation message.
5. Frontend displays a “Donation Posted Successfully” notification.

Figure 5

Sequence Diagram for Food Donation Workflow



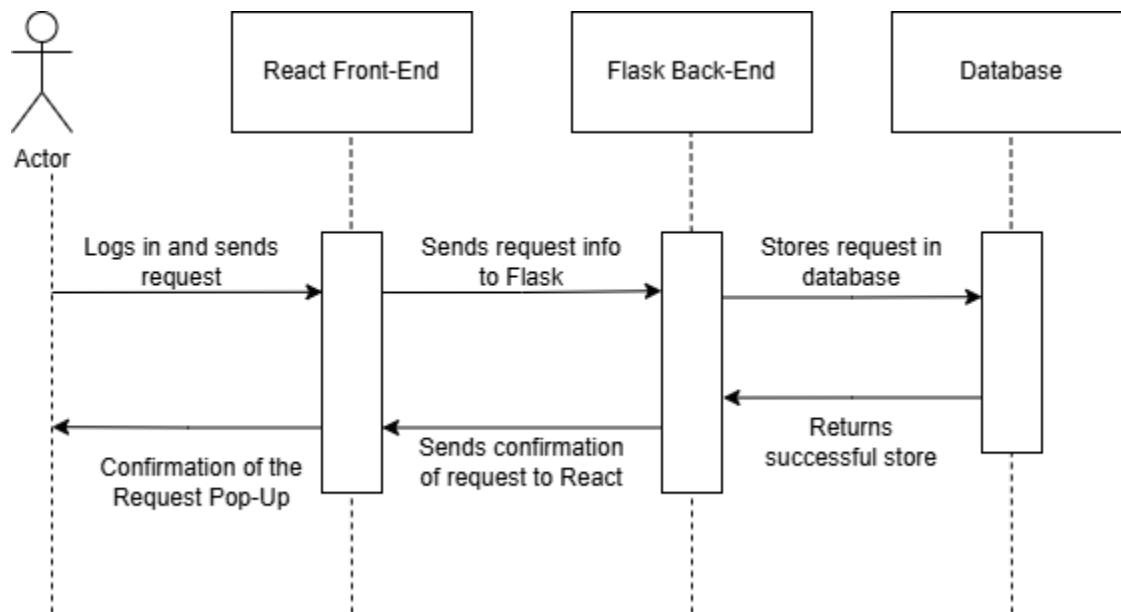
The sequence diagram below outlines the process of a donor posting a food request, showing how the system components interact.

Steps:

1. Donor logs in via the React frontend.
2. The frontend sends a POST request to the Flask API with request details.
3. Flask validates the input and stores it in the database.
4. Flask returns a confirmation message.
5. Frontend displays a “Request Sent Successfully” notification.

Figure 6

Sequence Diagram for Food Request Workflow



Below showcases how we plan to address security, mainly through the use of JSON Web Tokens and Database verification. JSON Web Tokens (JWTs) are cookies used for making sure no one can access unauthorized routes as well as making sure information is securely transferred

between two parties (Okta, n.d.). In our case, we plan to utilize JWTs to make sure a user can send their requests to the database securely and no one can intercept the requests. We also plan to use the JWTs as well as internal verification via the database to make sure a normal user or someone who is not logged in can't access admin-level controls such as approving the requests or modifying the users inside the database, and thus cannot access the database.

Figure 7

Sequence Diagram for a successful JWT Login

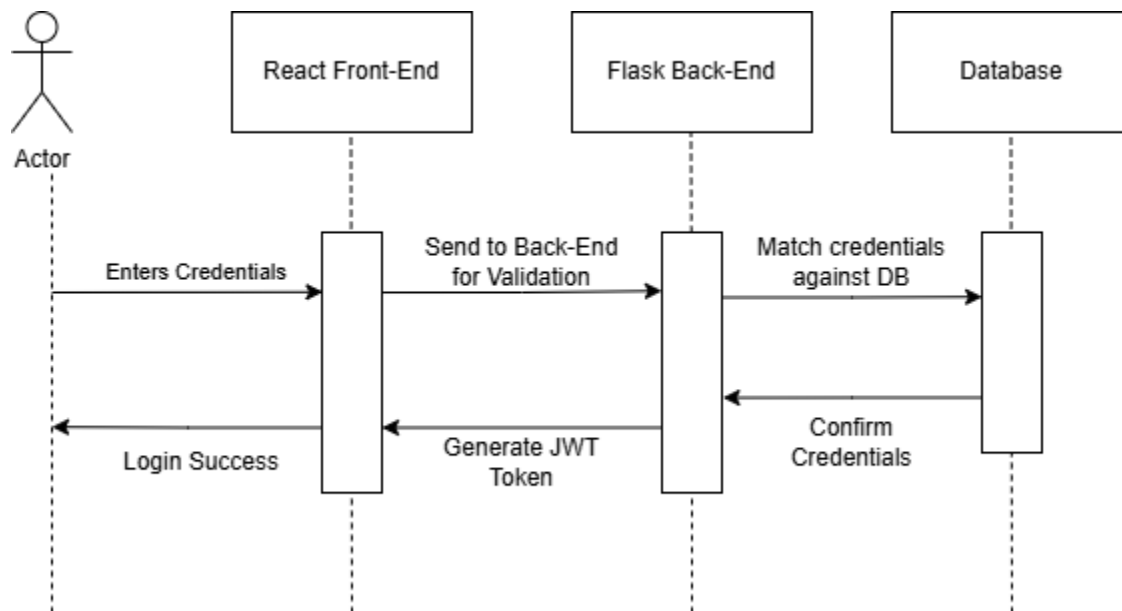
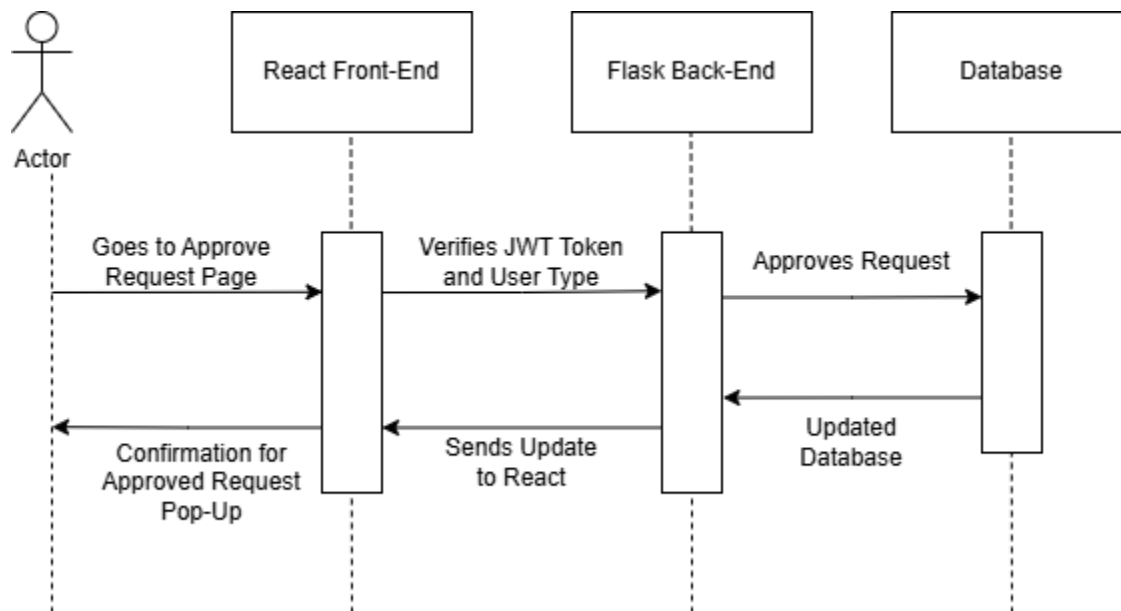
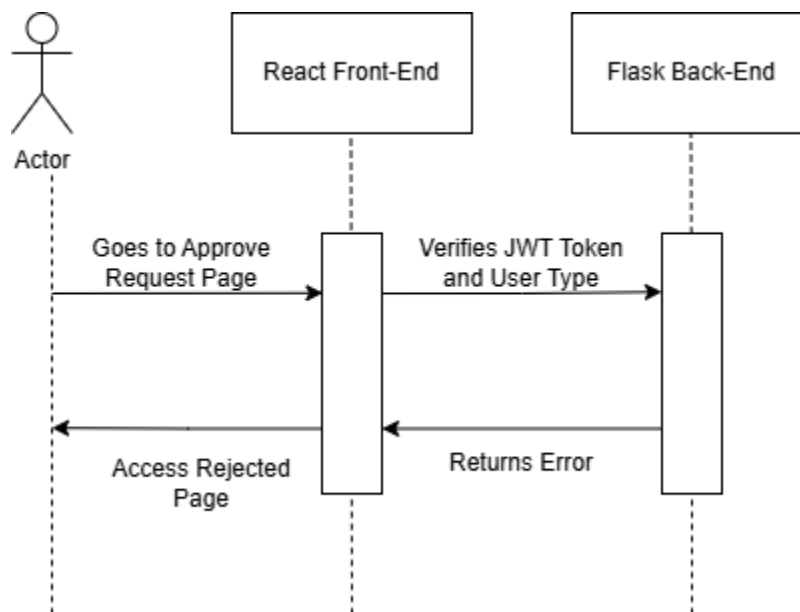


Figure 8

Sequence Diagram for an Admin Approving a Request

**Figure 9**

Sequence Diagram for Invalid Access to an Admin Page



Rationale Management

Rationale management captures the reasoning behind design decisions, alternatives considered, and the debates that led to consensus. This practice enhances traceability and accountability during software development (MacLean et al., 1991).

The following criteria guided the team’s decision-making throughout the design process:

Table 23

Criterion

Criterion	Description	Justification
Scalability	Ability of the system to handle growth in users and data.	Ensures FoodLink can support multiple food banks and donors.
Maintainability	Ease of updating and fixing components.	Simplifies future modifications as features evolve.
Usability	Interface clarity and accessibility for diverse users.	Vital for inclusivity among donors and recipients with varying technical skills.
Security	Protection of user data and prevention of unauthorized access.	Ensures confidentiality in donation and user records.
Performance	Efficient processing of transactions and API calls.	Reduces latency for real-time donation matching.

Framework Selection

- Issue: Selecting a frontend framework.
 - Alternatives Considered: React, Vue, Angular.
 - Decision: The team selected React.

- Rationale: React's reusable component structure and strong ecosystem made it ideal for rapid iteration and long-term scalability.
- Criteria: Learning curve, scalability, integration ease, and documentation.

Backend Framework

- Issue: Choosing a backend framework for the application.
 - Alternatives Considered: Django, Flask, Node.js.
 - Decision: Flask was selected.
 - Rationale: Flask offers a lightweight structure and flexibility suited for RESTful APIs. It integrates well with Python, which matched the team's skill set and reduced setup complexity compared to Django.
 - Criteria: Lightweight setup, ease of integration, learning curve, and speed of development.

Database Management

- Issue: Choosing a database system for storing user and donation data.
 - Alternatives Considered: MySQL, PostgreSQL, MongoDB.
 - Decision: MySQL was chosen.
 - Rationale: MySQL's relational model and reliability support transactional operations and ensure data integrity for donor–recipient records. It is widely supported and integrates easily with Flask.
 - Criteria: Reliability, ease of use, relational structure, and integration compatibility.

System Architecture

- Issue: Determining the overall architecture for the system.
 - Alternatives Considered: Monolithic architecture, microservices, three-tier architecture.
 - Decision: Adopted a three-tier architecture (presentation, application, and data layers).
 - Rationale: This model improves scalability, separates concerns, and facilitates independent updates of UI, logic, and data components.
 - Criteria: Scalability, maintainability, modularity, and clarity of responsibilities.

Security and Authentication

- Issue: Selecting an authentication method to secure access and data exchange.
 - Issue: Selecting an authentication method to secure access and data exchange.
 - Alternatives Considered: Session-based authentication, OAuth2, JSON Web Tokens (JWT).
 - Decision: JWT (JSON Web Tokens) was chosen.
 - Rationale: JWT allows stateless authentication and secure transmission of user credentials between frontend and backend, reducing server-side session management overhead.
 - Criteria: Security, scalability, compatibility with APIs, and implementation simplicity.

Collectively, these rationale records ensure that future developers can understand why specific design choices were made, not just what was implemented aligning with industry best practices in software design documentation (Pressman & Maxim, 2020).

Verification

In this section, we describe the verification process for FoodLink and the methods used to ensure the system aligns with its specified requirements. We also summarize the system-level test cases and their traceability to user stories.

Definition of Verification

In software engineering, *verification* refers to the set of activities used to evaluate whether a software system correctly implements the requirements defined during analysis and design. Sommerville (2016) explains that verification answers the question, “*Are we building the product right?*”, emphasizing consistency, correctness, and completeness with respect to documented specifications. Pressman and Maxim (2020) further describe verification as ensuring that the software “correctly implements a specific function,” highlighting its role in examining accuracy and adherence to technical expectations before the system is validated by end users. Thus, verification focuses on confirming that the system behaves according to its intended design, independent of user perception or real-world use.

Verification Process

Our team followed an iterative verification process aligned with Agile Scrum practices. Verification activities were conducted at the end of each sprint and again during a final, comprehensive system-level testing phase. The process included the following steps:

1. Review of User Stories and Acceptance Criteria

The team ensured that all user stories were clear, measurable, and testable before designing test cases.

2. Development of System-Level Test Cases

Test cases were created to evaluate end-to-end system behavior. Because the assignment does not require unit testing, the focus remained on full workflow validation for major functionalities such as registration, login, donation posting, food requests, and administrative actions.

3. Execution of Planned System Tests

Tests were executed in a controlled environment to evaluate system flows from the user's perspective. Each test verified proper handling of inputs, expected outcomes, and error conditions.

4. Defect Logging and Resolution

Any unexpected results or inconsistencies were documented and reviewed. Critical issues were prioritized for immediate correction, while minor issues were scheduled for follow-up in later sprints.

5. Re-testing and Verification Closure

Once issues were addressed, test cases were re-executed to confirm expected behavior.

This verification process ensured alignment with functional requirements, improved system reliability, and strengthened traceability between requirements, design, and testing activities.

System Level Test Cases

The following system-level test cases were developed to validate the functional flows of FoodLink. These test cases focus on assessing whether the system, once implemented, would perform according to the expected behavior defined in the user stories and acceptance criteria.

Table 24

Test Cases

TC ID	Description	Preconditions	Steps	Expected Result	Actual Result
TC-01	Register a new user	User not registered	1. Navigate to registration page 2. Enter required information 3. Submit form	Account is created and confirmation email is generated	Passed but does not send email currently
TC-02	Login with valid credentials	User account exists	1. Go to login page 2. Enter username and password 3. Submit	User is authenticated and redirected to profile	Passed Correctly
TC-03	Login with invalid credentials	None	1. Enter incorrect login data 2. Submit	Error message appears indicating invalid credentials	Passed Correctly
TC-04	Reset password	Registered account exists	1. Select “Forgot Password” 2. Enter registered email 3. Submit	Password reset instructions sent via email	Passed but does not send email currently
TC-05	Update user profile	User logged in	1. Access profile page 2. Edit information 3. Save	Profile updates successfully	Passed Correctly
TC-06	Create a food donation	Donor logged in	1. Access donation page 2. Enter donation details	Donation appears in pending review for admin	Passed correctly

			3. Submit		
TC-07	Edit a food donation	Donor has existing donation	1. Access “My Donations” 2. Select donation 3. Edit 4. Save	Donation updates successfully	Partially implemented
TC-08	View available food	Recipient logged in	1. Go to “Available Food” 2. Browse list	Available food items displayed	Passed correctly
TC-09	Filter food items	Donations of varying categories exist	1. Open filters 2. Select categories	Only matching food items appear	Passed Correctly
TC-10	Request a food item	Food item available	1. Select donation 2. Confirm request	Food is allocated to recipient	Not implemented
TC-11	Approve donation	Admin logged in	1. Open Pending Donations 2. Approve request	Donation becomes visible to recipients	Passed correctly
TC-12	Reject donation	Donation pending	1. Review donation 2. Reject	Donation removed; donor notified	Passed correctly
TC-13	Deactivate user	User exists	1. Access User List 2. Select user 3. Deactivate	User account is disabled	Partially implemented

Traceability Between the Test Cases/Plan and User Story

The following table was created in order to match the user story across the multiple test cases that were developed.

Table 25

Traceability of User Stories & Test Cases

User Story Feature	User Story Description	Related Test Cases
Feature 1: Registration	Create account, verify email, handle invalid inputs, receive confirmation	TC-01, TC-03
Feature 2: Login	Login, incorrect login, reset password, save login	TC-02, TC-03, TC-04
Feature 3: Profile	View and edit profile	TC-05
Feature 4: Food Donation	Create donation, edit donation	TC-06, TC-07
Feature 5: Food Requests	View available food, filter food, request food	TC-08, TC-09, TC-10
Feature 6: Admin Panel	Approve donations, reject donations, deactivate users	TC-11, TC-12, TC-13

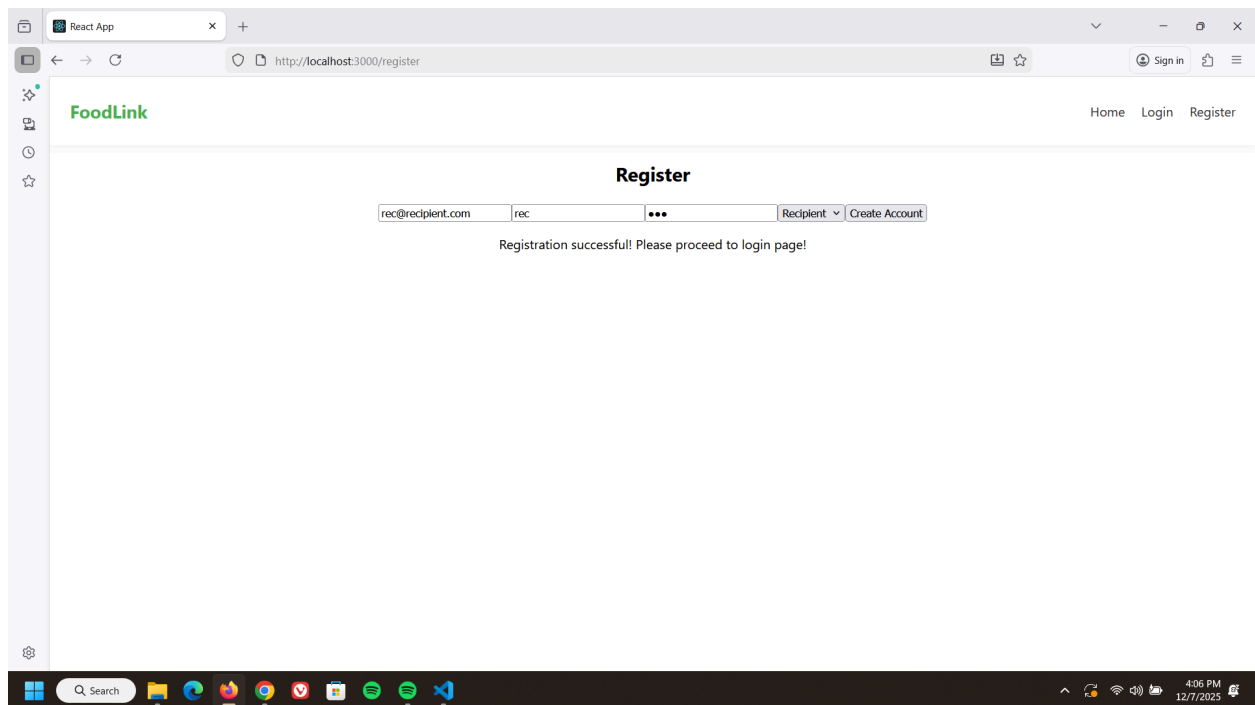
Test Case Results:

TC-01

For test case 1, registering a new user, the test passes correctly.

Figure 10

Successful Registration

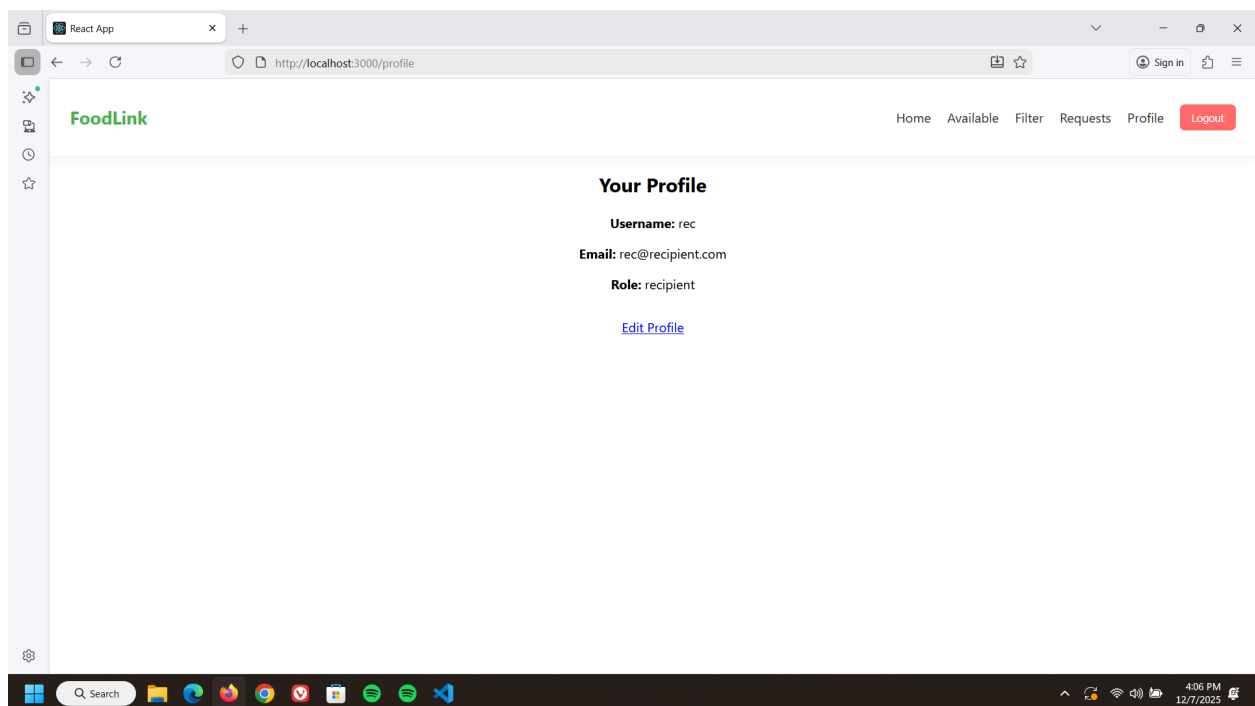


TC-02

For test case 2, logging in with valid credentials, the test passes correctly.

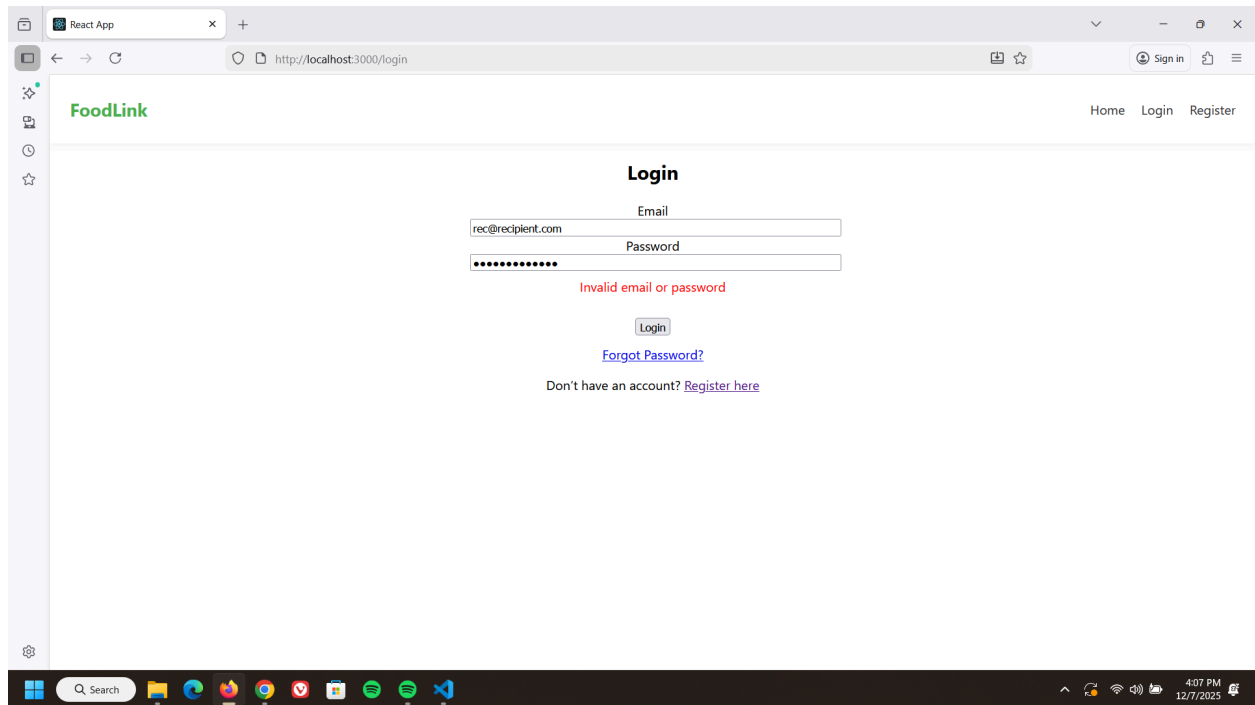
Figure 11

Successful Login



TC-03

For test case 3, trying to login in with invalid credentials, the test passes correctly.

Figure 12*Failed Login***TC-04**

For test case 4, resetting your password, the test passes correctly.

Figure 13*Obtaining Reset Link*

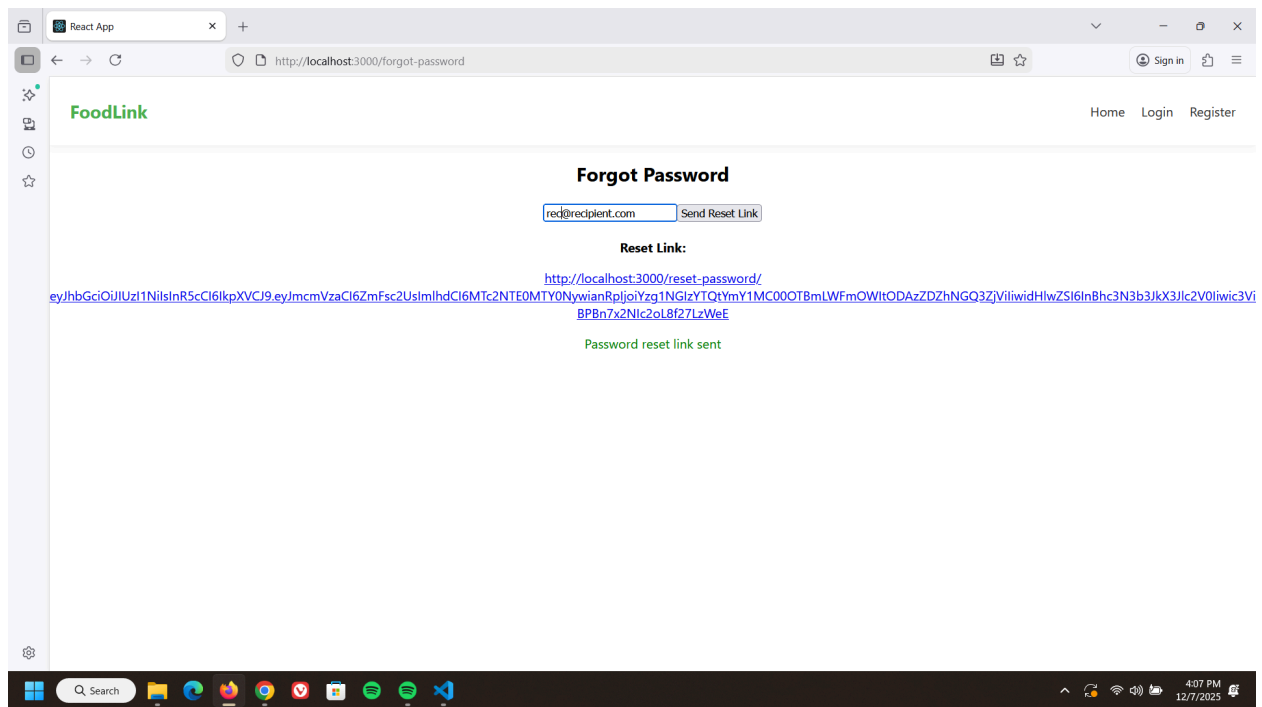
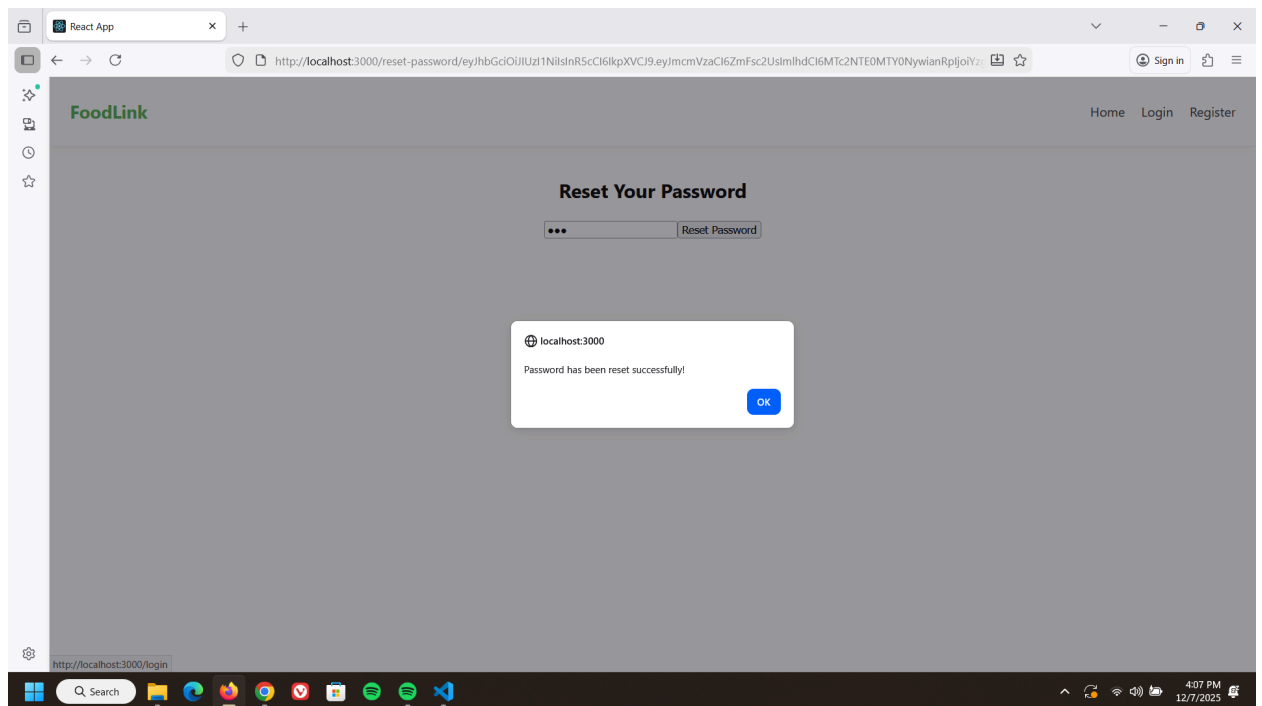


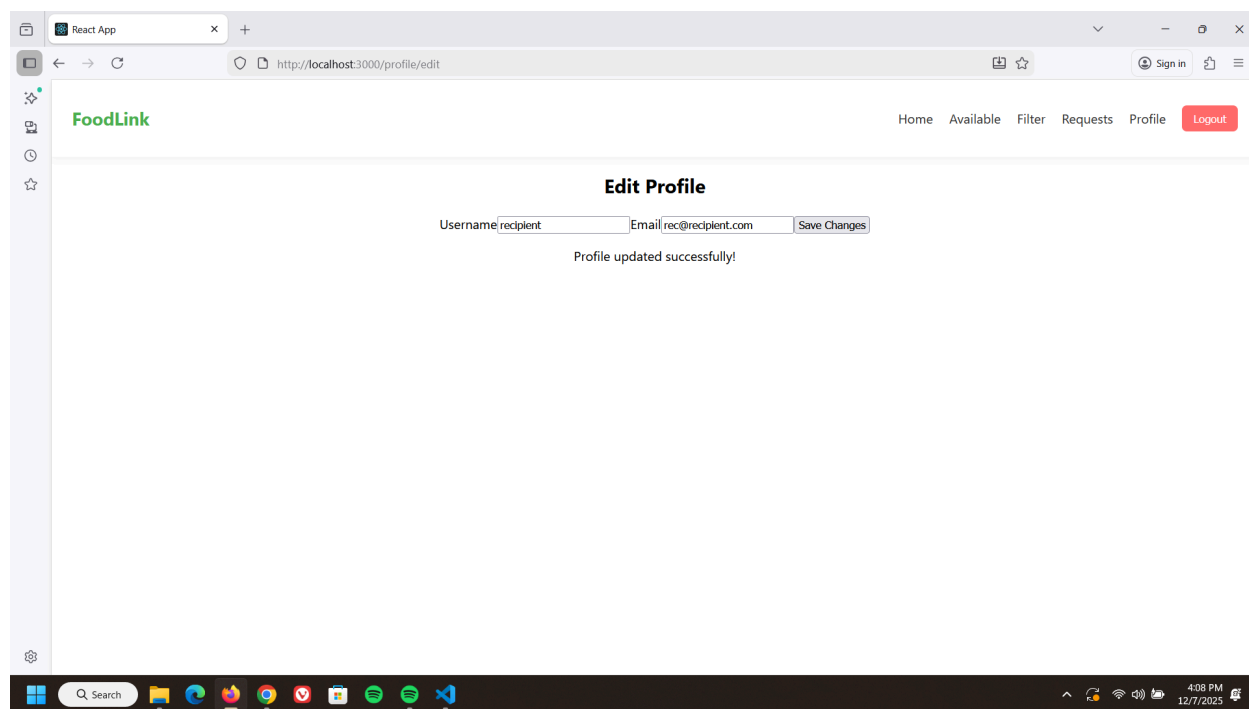
Figure 14

Successful Reset



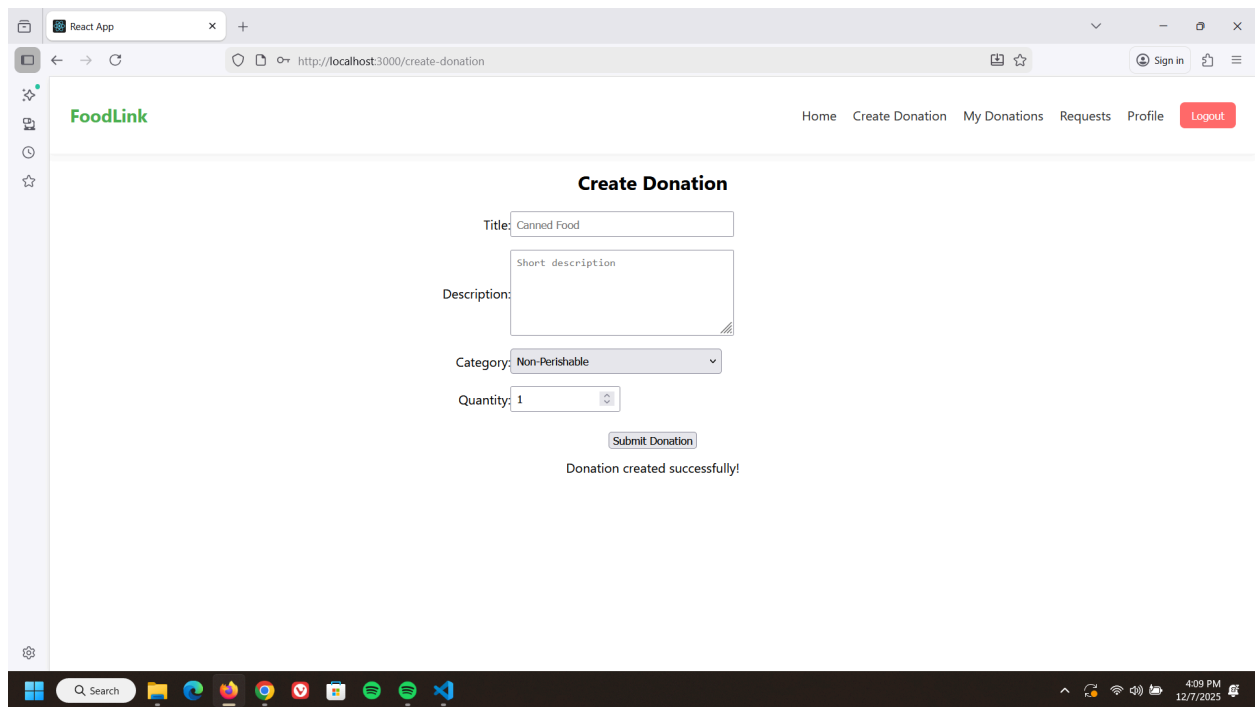
TC-05

For test case 5, updating a user's profile, the test passes correctly.

Figure 15*Updating Profile****TC-06***

For test case 6, creating a food donation as a donor, the test passes correctly.

Figure 16*Successful Food Donation Creation*

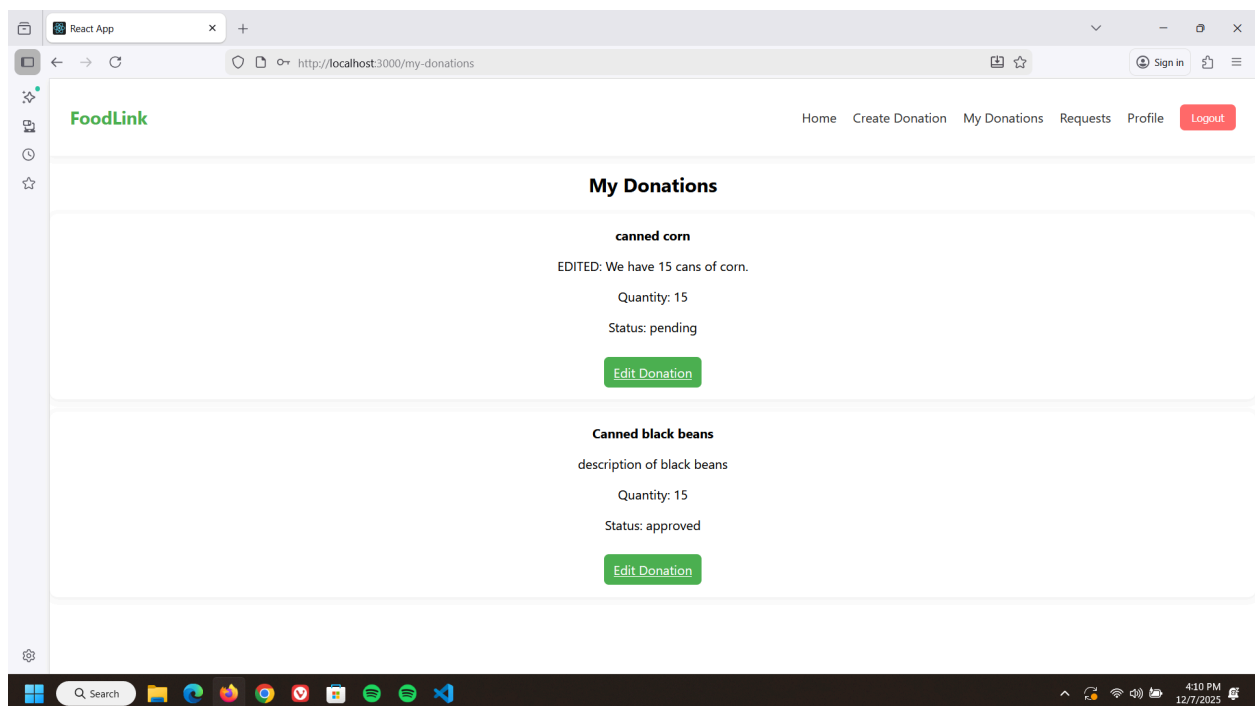


TC-07

For test case 7, editing a donation, the test passes correctly.

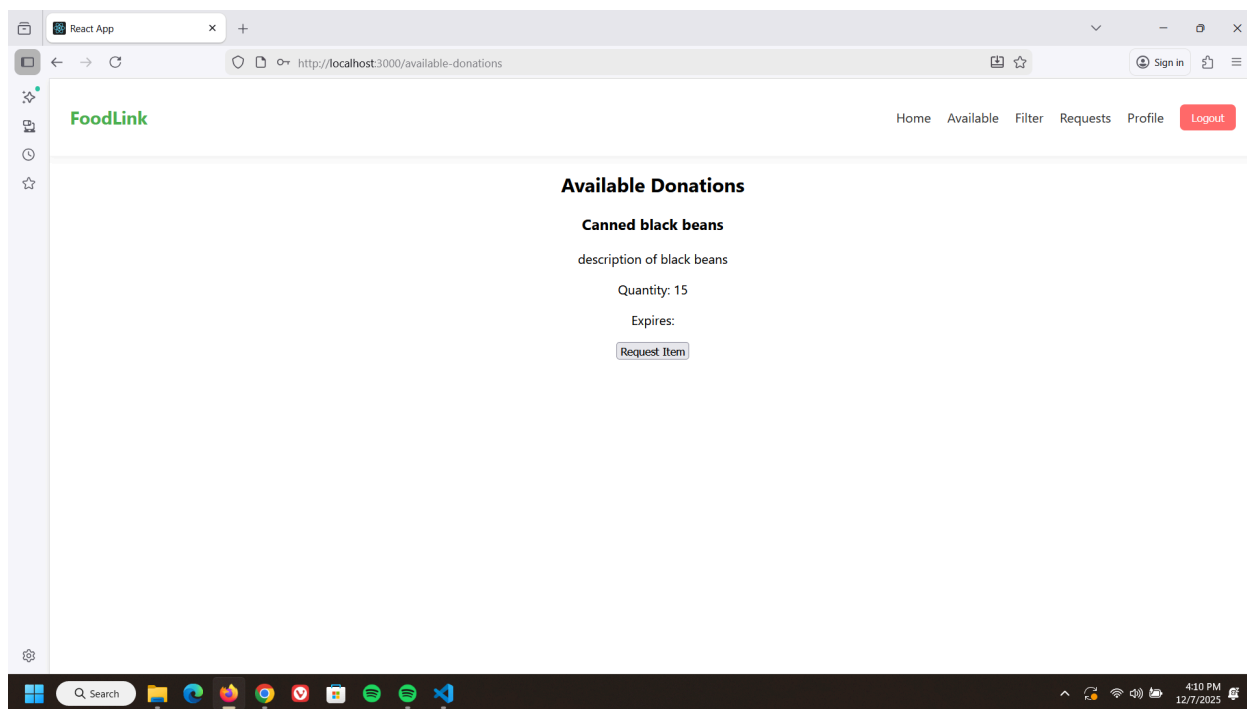
Figure 17

Successful Edited Donation



TC-08

For test case 8, viewing available food/donations, the test passes correctly.

Figure 18*Viewing Available Donations***TC-09**

For test case 9, filtering food items, the test passes correctly.

Figure 19

Filter Page

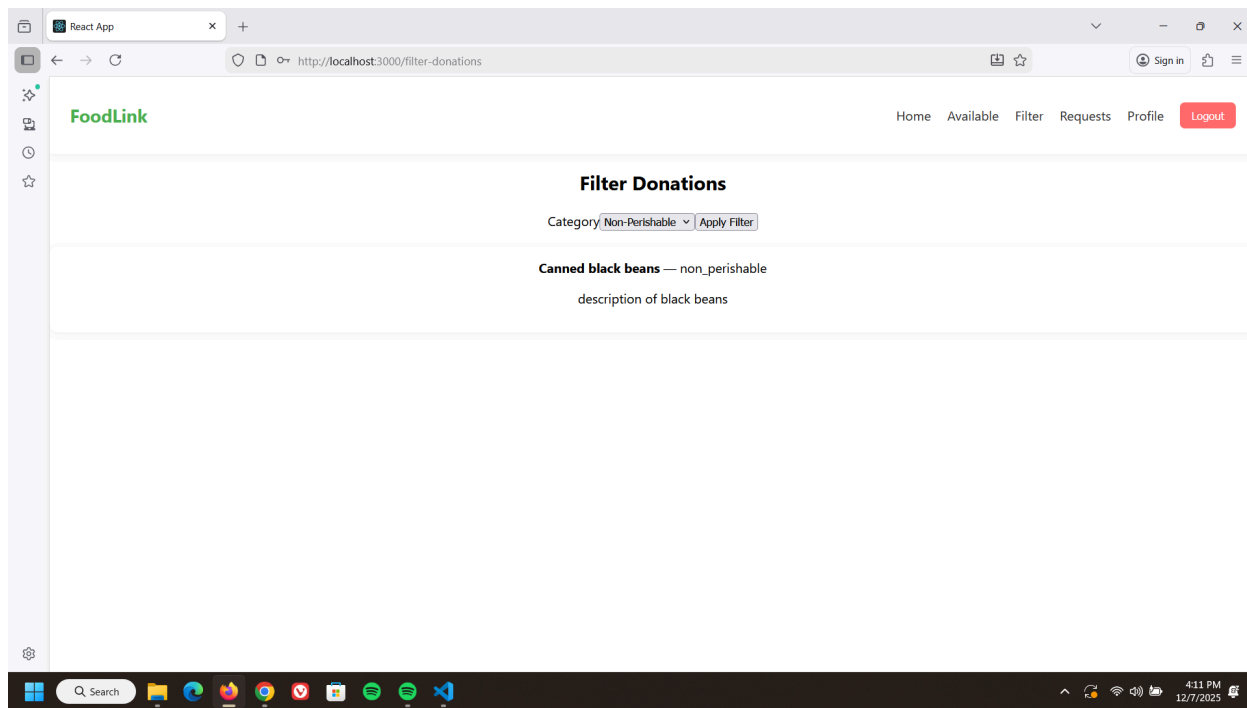
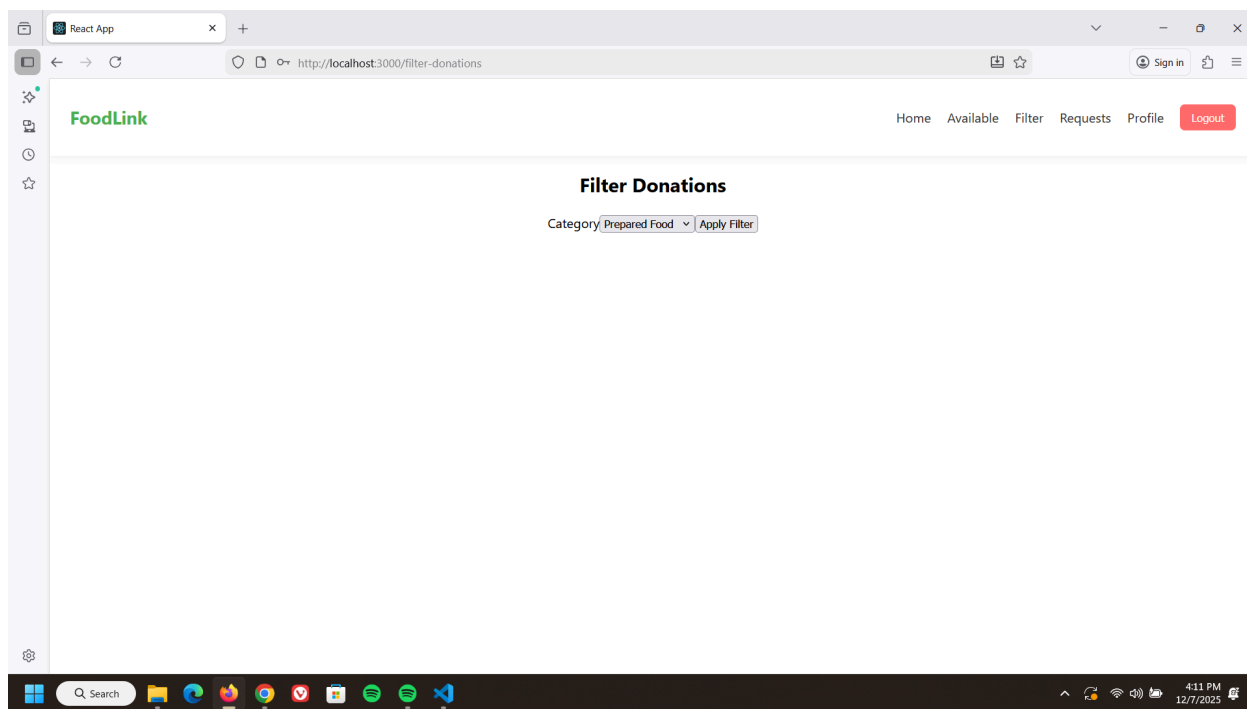


Figure 20

Change Filter



TC-10

For test case 10, requesting a food item, the test passes correctly.

Figure 21

Available Donations Page Before Request

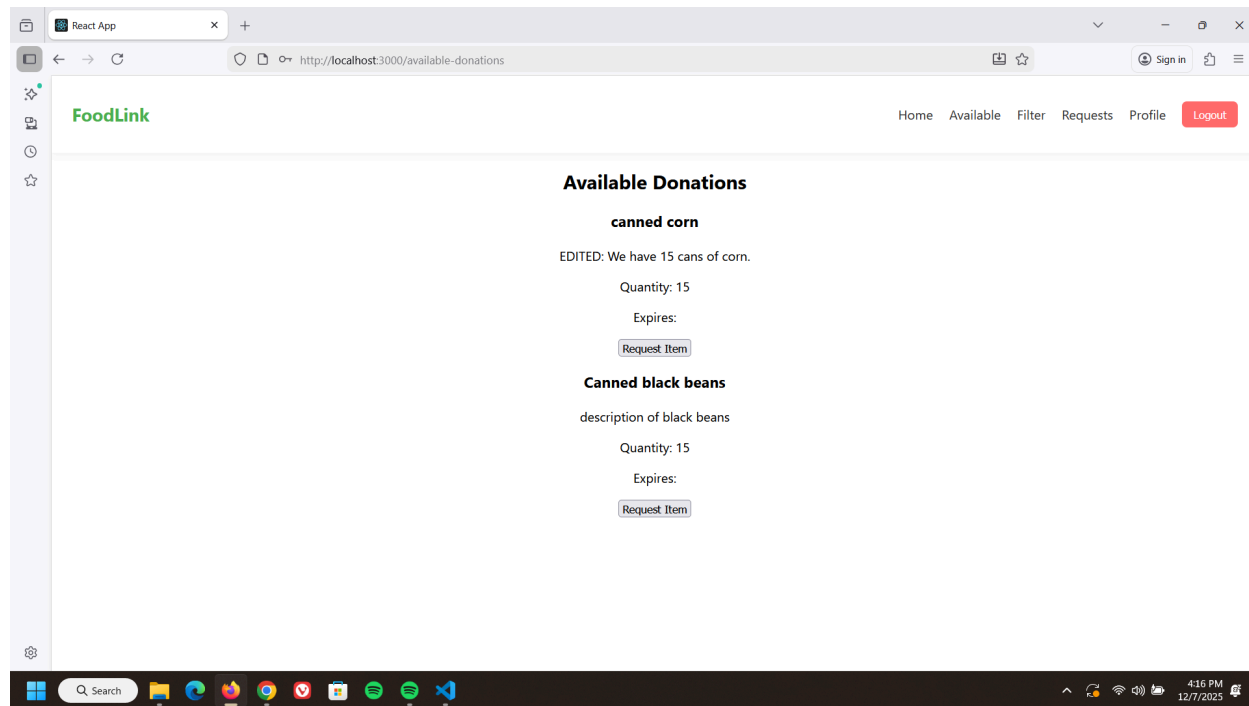
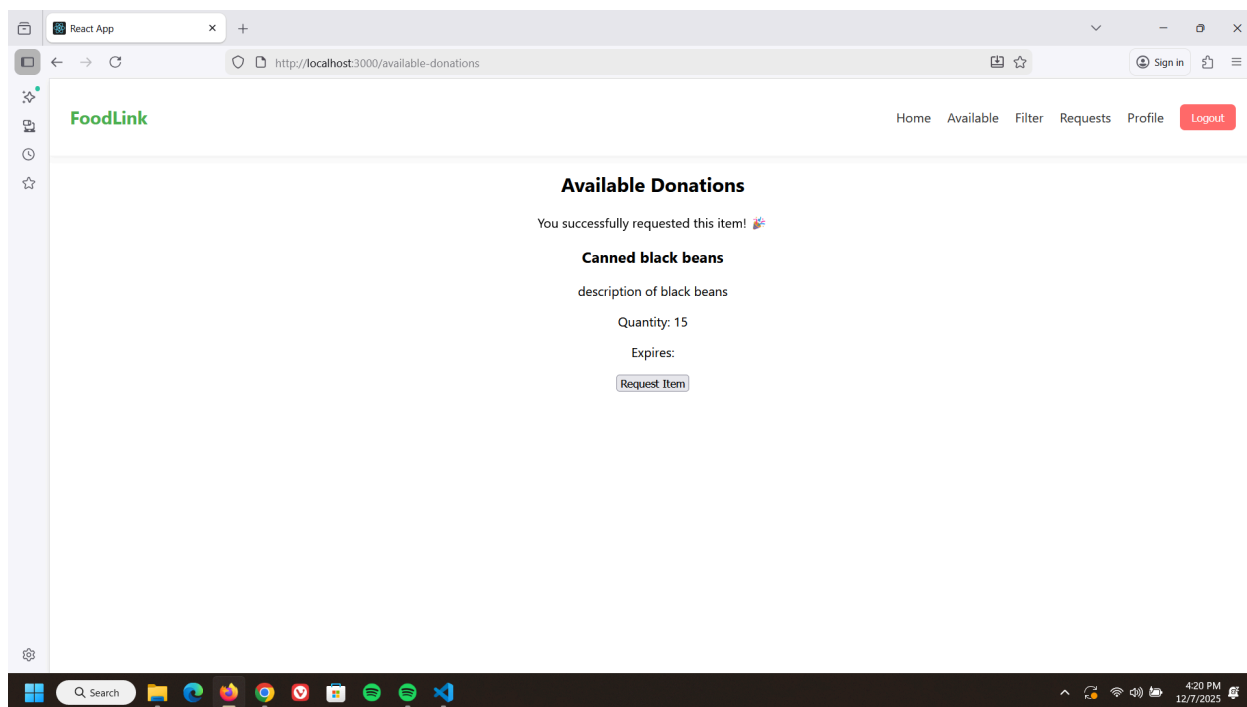


Figure 22

Successful Request



TC-11

For test case 11, approving a donation by an admin, the test passes correctly.

Figure 23

Admin Dashboard

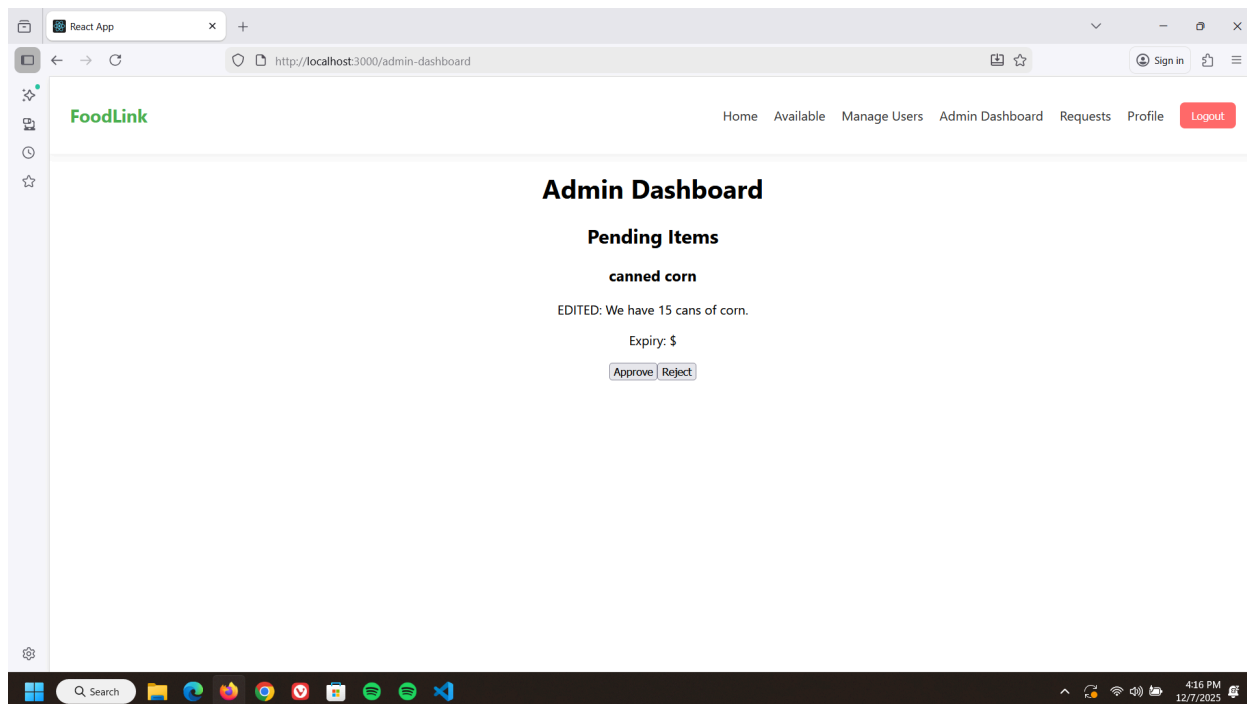
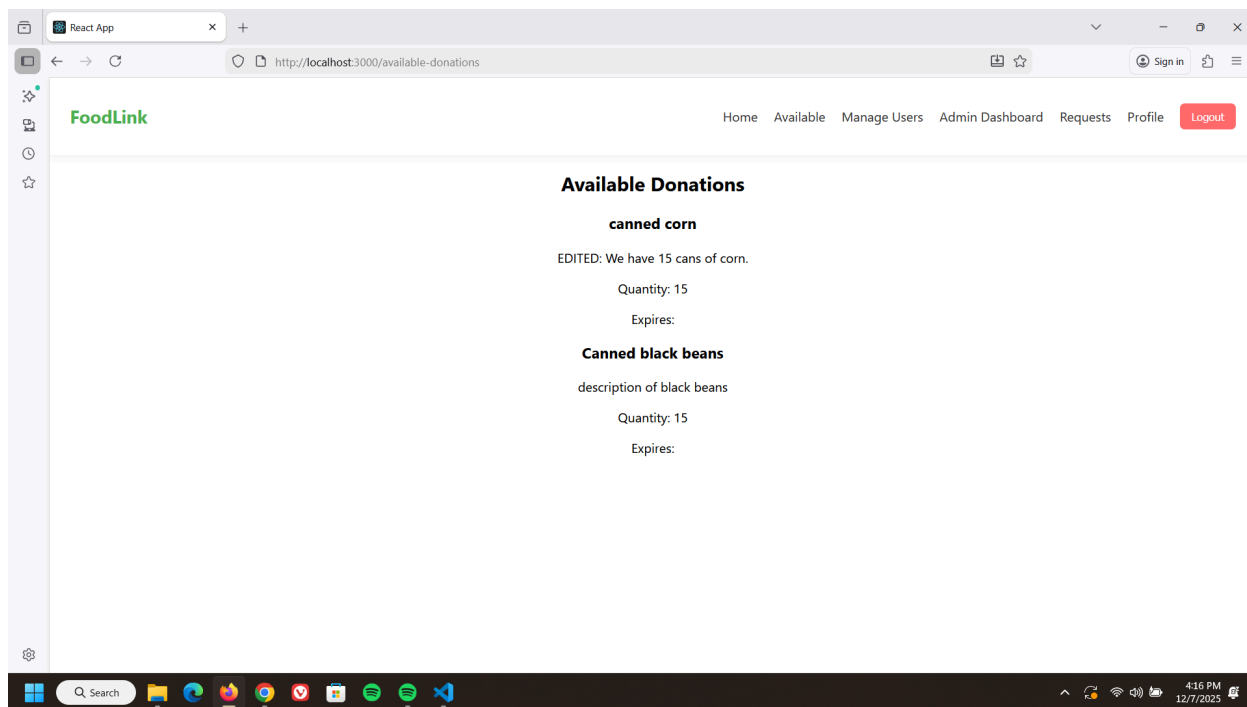


Figure 24

Now Available Donation



TC-12

For test case 12, rejecting a donation, the test passes correctly.

Figure 25

Bad Donation Pending

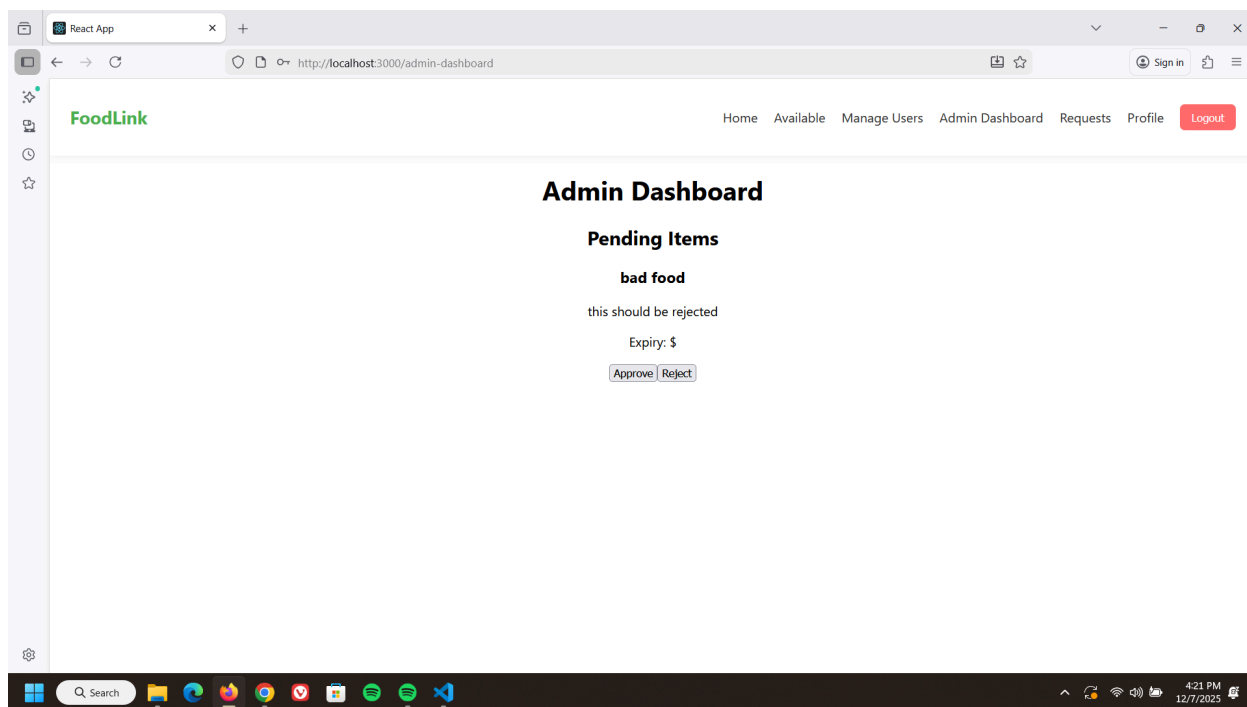
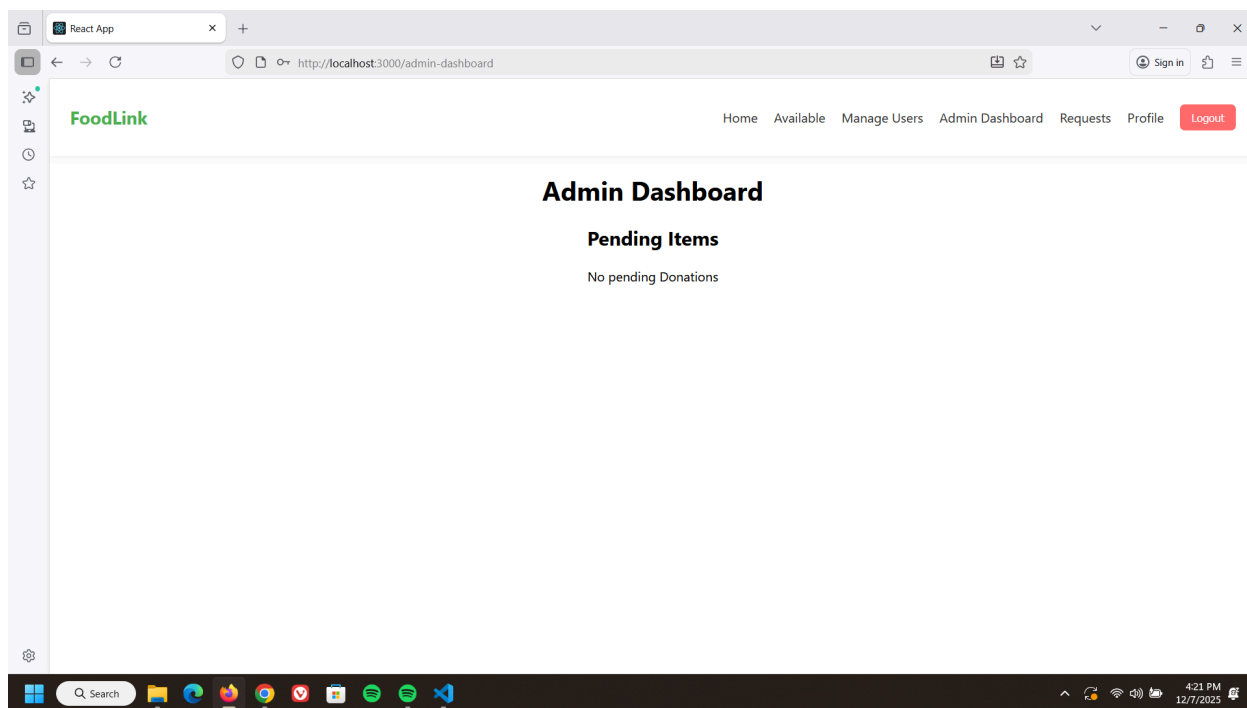


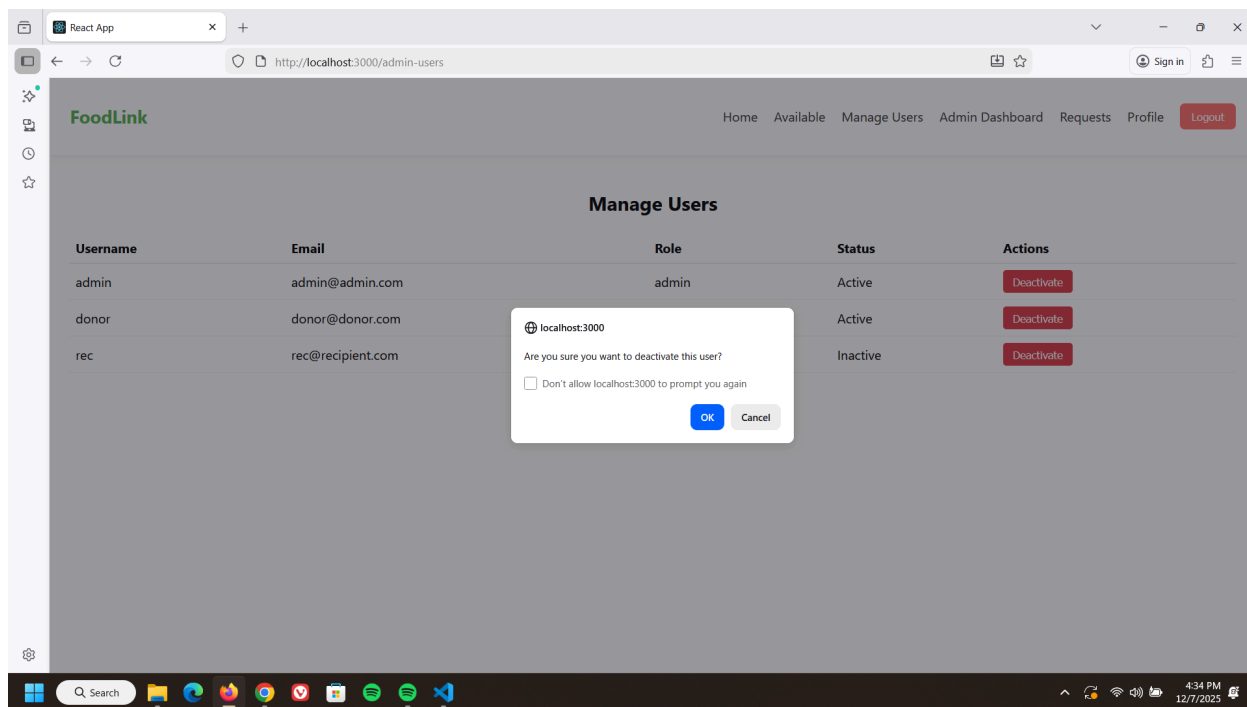
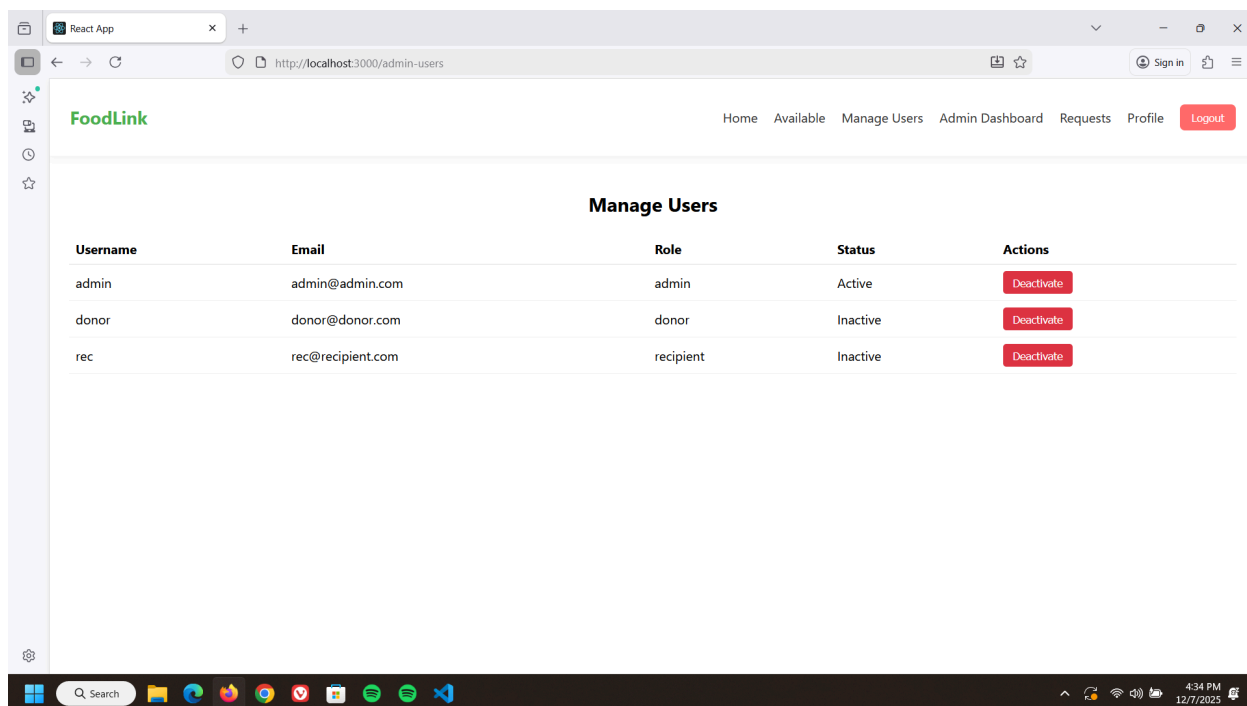
Figure 26

Admin Dashboard After Rejecting



TC-13

For test case 13, deactivating a user, the test passes correctly.

Figure 27*Deactivating a Donor***Figure 28***Dashboard Update*

Reflection**Table 26***Actual Timeline*

Sprint	Duration	User Stories	Tasks
1	1 week (Complete)	Registration	- Decide on framework and tools (backend, frontend, database) - Set up project repository - Design and implement initial database schema - Build backend skeleton - Create simple “Hello World” frontend to verify setup
2	1 week (Complete)	Login	- Connect backend to database - Develop initial frontend skeleton (navigation, basic pages) - Implement user registration and login - Implement Security via JSON Web Tokens (JWT) - Add ability for donors to post donations (basic form) - Partition database for donors and recipients
3	1 week (Complete)	Food Donation	- Implement donation browsing for recipients (food banks, shelters, individuals)

			- Add filtering (by type, expiration, availability) - Develop claim/request functionality - Apply rules for individual donors (sealed/unopened items) - Add basic admin monitoring dashboard
4	1 week (Complete)	Food Requests	- Connect frontend and backend fully (JSON APIs) - Enable editing and canceling of donations - Implement fair-share distribution logic (balance among recipients) - UI polish (improve look and usability) - Perform full system testing - Prepare documentation and final demo
5	3 weeks (Incomplete)	Profile	- Implement profile dashboard (view and edit account details) - Add user role management (donor, recipient, moderator) - Build admin panel features (approve/reject donations, deactivate users) - Integrate notifications (confirmation emails, status updates) - Final integration of frontend and backend

			- Conduct end-to-end testing - Polish UI/UX and accessibility - Finalize documentation and prepare presentation/demo
--	--	--	--

Table 27

Comparison Proposed vs Actual Timeline

Sprint	Proposed	Actual	Result
1	2 weeks	1 week	Met
2	2 weeks	1 week	Met
3	2 weeks	1 week	Met
4	2 weeks	1 week	Met
5	2 weeks	3 weeks	Longer and Incomplete

In the early stages of the project, we were able to meet most of the deadlines outlined in the proposed timeline so we shortened the sprint durations. However, as implementation progressed, we encountered several unexpected bugs later on and some features required significantly more time than anticipated.

Certain tasks demanded additional research to determine the most effective approach for implementation, which further extended our schedule. Throughout each sprint, our priority was to complete the core features necessary for the overall functionality of the system. As such, we ended up pushing back some features in favor of working on others. We selected lower-complexity tasks to ensure steady progress and maximize the number of completed items within each sprint.

The two major cases we encountered were specifically with our Food object class and the Request flow. As we created the app, we kept encountering issues with the Food object class when trying to access it or grabbing data from it, and as such, scrapped it in terms of just having a donation object which would contain the food item and its details inside it. From there, each donation was assigned an id and that was how it was communicated with. As for the request flow, we realized it required an entire new table in the database and would increase the complexity of interacting with donors and as such, kept it to simply just letting recipients claim donations from the available donations page for now as time didn't permit for us to rework the entire database.

In general, when a sprint depended on a feature or deliverable that was not finished in a previous cycle, we reallocated that task to a later sprint to maintain workflow continuity and avoid blocking development. This ended up causing some problems because when we reached the last sprint we realized we had pushed back a lot of stuff that would require major reworkings, such as the Food/Request features mentioned earlier. This ended up slowing down our progress significantly as we started working on trying fixes for everything, however those two were the two main issues that ended up staying incompletd.

As we progressed through the sprints, we also started planning out more features that didn't end up being implemented such as sending messages between donors and recipients when accepting a donation or accepting a request but did not have time to implement that either. We also had some partial implementations of new features such as re-activating users after de-activation. In the end, we were still able to accomplish a lot of the features we planned as well as pass all the current user story/test cases we had planned out.

References

- Feeding America. (n.d.) *Meal Connect: How It Works*. <https://mealconnect.org/how-it-works>
- Okta. (n.d.) *Introduction to JSON Web Tokens*.
<https://www.jwt.io/introduction#what-is-json-web-token>
- U.S. Department of Agriculture. (n.d.). *Food Waste FAQs*.
<https://www.usda.gov/about-food/food-safety/food-loss-and-waste/food-waste-faqs>
- U.S. Department of Agriculture (2020, August 13). *Good Samaritan Act Provides Liability Protection for Food Donations*.
<https://www.usda.gov/about-usda/news/blog/good-samaritan-act-provides-liability-protection-food-donations>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- MacLean, A., Young, R. M., Bellotti, V. M., & Moran, T. P. (1991). *Design rationale: The argumentation basis of design*. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems* (pp. 203-208). ACM.